ESTUDO DIRIGIDO · M3 · AULA 2 DE 3 · M3 · WORKFLOWS, AGENTES, TOOL USE E MCPS

Tool use: o modelo pede, o seu código executa

Ao fim deste estudo você acha cada pedido e cada resultado num registro do Claude Code e escreve e testa uma ferramenta com contrato: descrição clara, validação, erro que ensina e risco declarado.

Aula de Seg, 16/11/2026 | Tempo sugerido: cerca de 3 horas, em uma ou duas sentadas | Sistemas com IA para Devs · CESAR School

Como usar este estudo

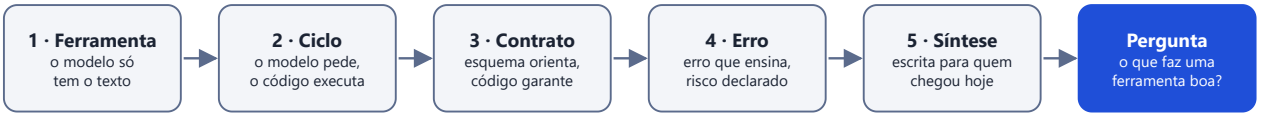
Primeiro leia os conceitos na ordem: cada um sai do anterior, como na aula. Depois faça as práticas, que repetem no seu computador o que o professor demonstrou, com o Claude Code e o caso Balcão. No fim, confira a autoavaliação e as respostas das perguntas.

ANTES DE COMEÇAR

- [ ] Claude Code instalado e com login feito. Teste no terminal: `claude -p "responda apenas: ok"` precisa responder `ok`.
- [ ] Python 3.11 ou mais novo e, na pasta `caso-balcao`, `pip install -r requirements.txt` (SDK do MCP) e `pip install pytest`.
- [ ] A pasta `caso-balcao` do repositório da turma, github.com/victorhwfreire/sistemas_ia_devs_b2c_20262, no seu computador, sem arquivos `.jsonl` em `estado`. Os comandos partem de `caso-balcao/ferramentas`, salvo quando o passo diz outra coisa.
- [ ] No Windows PowerShell, rode antes `[Console]::OutputEncoding = [Text.Encoding]::UTF8` para os acentos saírem certos.

O mapa da aula

Pergunta da aula: **o que faz uma ferramenta ser boa para um modelo?**



Mapa da aula: cinco conceitos em cadeia. Para o modelo, a ferramenta é só o texto que a descreve; no ciclo do function calling, o modelo pede e o código executa; no contrato, o esquema orienta e o código garante; o erro precisa ensinar e o risco precisa estar declarado; e a síntese é escrever a ferramenta para quem chegou hoje. Termina na pergunta destacada: o que faz uma ferramenta ser boa para um modelo?

Conceito 1 de 5

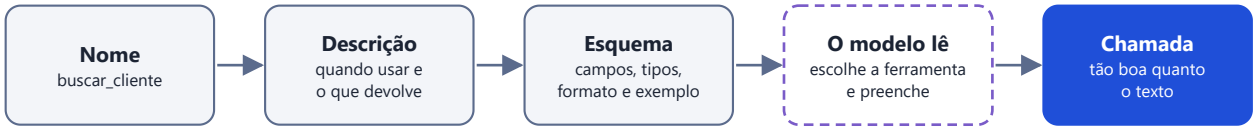
Para o modelo, a ferramenta é só o texto que a descreve

> **Ferramenta é nome, descrição e esquema de entrada; o modelo escolhe qual usar e preenche os argumentos só a partir desse texto.**

Quando você dá uma ferramenta a um modelo, não entrega a função: entrega três pedaços de texto. O nome; a descrição, que diz quando usar e o que devolve; e o esquema de entrada, em JSON Schema, com campos, tipos e, se você escrever, formato e exemplo. O modelo nunca vê o código: decide se chama a ferramenta, qual delas e o que pôr em cada campo lendo só esse texto.

A Anthropic descreve ferramenta como um contrato entre um sistema determinístico e um agente não determinístico: diante da mesma pergunta, o agente pode chamar a ferramenta, responder do que já sabe ou pedir mais informação. Por isso ela é escrita para o modelo, não para outro desenvolvedor.

A descrição é a interface. No mesmo texto, a Anthropic conta que o Claude Sonnet 3.5 chegou ao estado da arte no SWE-bench Verified depois de refinar as descrições das ferramentas.



O que o modelo recebe de uma ferramenta, em cinco etapas: o nome, como buscar_cliente; a descrição, que diz quando usar e o que devolve; o esquema de entrada, com campos, tipos, formato e exemplo; o modelo lê esse texto, escolhe a ferramenta e preenche os argumentos; e a chamada sai tão boa quanto o texto.

NO CASO BALCÃO

No contrato fraco do servidor do Balcão, `get_data(cliente, periodo)` tem a descrição “Consulta dados.”. No forte, `consultar_cobrancas(cliente_id, mes)` diz o que lista e quando usar, e traz formato e exemplo em cada campo (`CLI-102`, `2026-10`). Por baixo, as duas leem o mesmo `cobrancas.json`.

CONFUSÃO FREQUENTE

Achar que o modelo entende a ferramenta pelo código. Ele só recebe o nome, a descrição e o esquema publicados.

PARA CONFERIR · RESPOSTA NO FIM

O TCK-002 traz o nome do comércio, não o id. Com o contrato fraco, de onde o modelo tiraria o formato do campo `cliente`?

Conceito 2 de 5

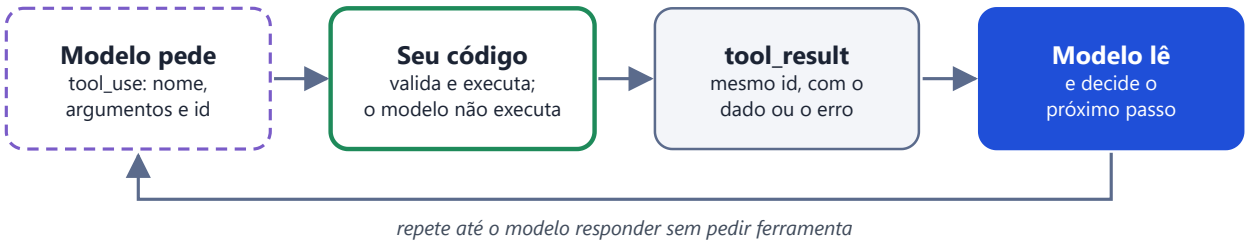
O modelo pede a ferramenta; quem executa é o seu código

> **No function calling, o modelo devolve um pedido `tool_use` com nome, argumentos e id; o seu código valida, executa e devolve um `tool_result` com o mesmo id; o modelo continua.**

Numa chamada à API com ferramentas, o modelo pode parar e devolver um bloco `tool_use` com `name` (a ferramenta), `input` (os argumentos em JSON) e `id`; na API da Anthropic, a resposta vem com `stop_reason` igual a `tool_use`. O seu programa executa a função e devolve um bloco `tool_result` com `tool_use_id` igual ao id e o resultado, ou o erro. Então chama o modelo de novo, com tudo isso na janela.

É o loop do agente da aula 1 por dentro: cada volta é um par `tool_use` e `tool_result`. O id casa cada resultado com o seu pedido quando o modelo pede várias ferramentas na mesma volta.

O modelo nunca executa nada. Toda ação passa pelo seu código: isso dá controle (validar, registrar, negar) e responsabilidade (sem resultado, o modelo decide às cegas). No Claude Code, cada pedido de ferramenta MCP vai ao servidor MCP, que executa a função; como esse protocolo funciona é a aula 3.



Ciclo do function calling: o modelo emite um tool_use com nome, argumentos e id; o seu código valida e executa, porque o modelo não executa nada; o resultado volta como tool_result com o mesmo id, trazendo o dado ou o erro; o modelo lê e decide o próximo passo; e o ciclo se repete até o modelo responder sem pedir ferramenta.

NO CASO BALCÃO

Num registro de `rodar_agente.py --modo mcp`, um bloco `tool_use` traz, por exemplo, `name` igual a `mcp__balcao__buscar_cliente` e `input` igual a `{"nome": "Pilates"}`; o `tool_result` seguinte traz o mesmo valor em `tool_use_id`. O servidor registra a chamada em `estado/chamadas.jsonl`.

CONFUSÃO FREQUENTE

Achar que o modelo roda a ferramenta. Ele só pede; sem o `tool_result`, fica sem o dado e pode inventar.

PARA CONFERIR · RESPOSTA NO FIM

Se o modelo pedir duas ferramentas na mesma volta, como o seu código diz qual resultado é de qual pedido?

Conceito 3 de 5

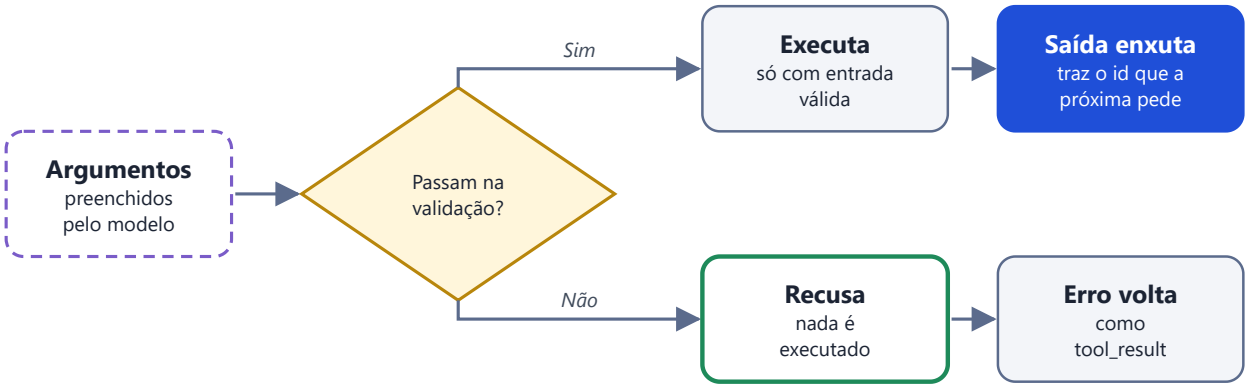
O esquema orienta; a validação no código garante

> **Contrato é o acordo de entrada e saída: o esquema diz o formato, o código confere antes de executar e a saída traz, estável e enxuta, o que o próximo passo precisa.**

O esquema orienta o modelo com tipo, formato e exemplo, mas é uma dica forte, não uma trava: o modelo gera texto. A validação mora no código que executa: conferir o formato, conferir se o registro existe e só então tocar nos dados. Mesmo com o modo estrito de algumas APIs, que garante o formato dos argumentos, a regra de negócio continua no código.

A outra ponta é a saída: estável, sempre com os mesmos campos, e enxuta, porque tudo o que a ferramenta devolve entra na janela e é reenviado nas voltas seguintes. A Anthropic recomenda campos com significado e controle de tamanho (paginação, filtro, truncagem); o Claude Code limita a saída de uma ferramenta MCP a 25 mil tokens por padrão.

Um bom sinal de contrato é o encaixe: a saída de uma ferramenta traz exatamente o campo que a próxima pede.



Decisão no código para os argumentos que o modelo preencheu: passam na validação? Se sim, a ferramenta executa só com entrada válida e devolve uma saída enxuta, que traz o id que a próxima ferramenta pede. Se não, recusa sem executar nada, e o erro volta ao modelo como tool_result.

NO CASO BALCÃO

`consultar_cobrancas` confere `cliente_id` com `CLI-\d{3}` e `mes` com `\d{4}-\d{2}`, confere se o cliente existe e só então lê os lançamentos. Devolve id, data, tipo, valor e status de cada um, mais o total. `buscar_cliente` devolve o `cliente_id` que `consultar_cobrancas` pede.

CONFUSÃO FREQUENTE

Confiar só no esquema. Ele não impede um argumento fora do formato nem um id que não existe; o código impede.

PARA CONFERIR · RESPOSTA NO FIM

Por que `buscar_cliente` devolve só id, nome, plano e mensalidade, e não o cadastro inteiro?

Conceito 4 de 5

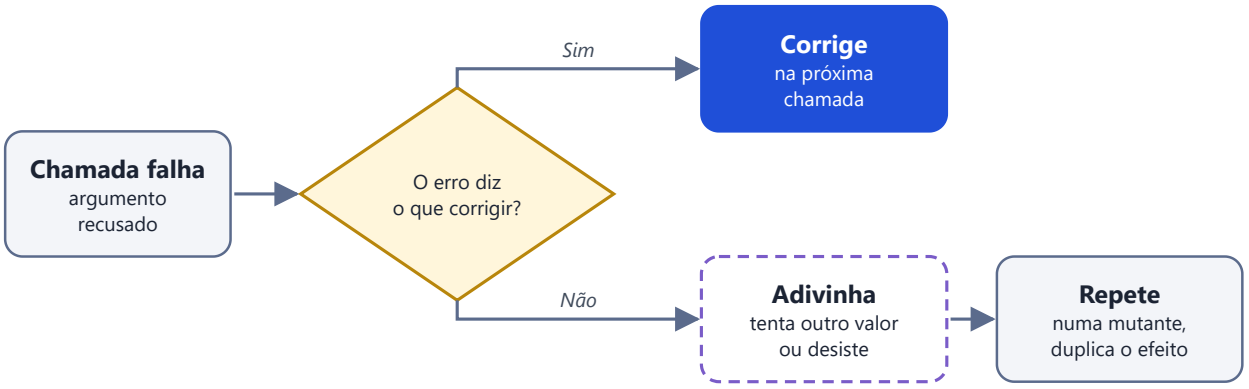
O erro precisa ensinar, e o risco precisa estar escrito

> **Mensagem de erro que diz o esperado, o recebido e o que fazer permite a autocorreção; e cada ferramenta declara sua classe de risco: leitura, mutante ou destrutiva.**

Quando o código recusa um argumento, o erro volta como `tool_result` e é a única pista do modelo. Um erro opaco, como `invalid input`, diz que algo deu errado, mas não o quê: o modelo tenta outro valor, repete ou desiste. Um erro que ensina diz o esperado, o recebido e o que fazer, como recomenda a Anthropic: a correção específica, em vez de códigos opacos ou tracebacks.

Adivinhar é repetir chamadas, e o custo depende da classe de risco. Leitura: repetir gasta token, nada mais. Mutante: muda estado; se não é idempotente, repetir duplica o efeito. Destrutiva: faz o que não se desfaz, como um estorno.

Por isso a classe precisa estar escrita: na descrição, que o modelo lê, e em metadados, que o sistema lê. No MCP, as anotações `readOnlyHint`, `destructiveHint` e `idempotentHint` cumprem esse papel; são dicas, não travas. A trava, um hook que confere a política antes do estorno, é a aula 3.



Decisão depois de uma chamada recusada: o erro diz o que corrigir? Se sim, o modelo corrige na próxima chamada. Se não, adivinha, tentando outro valor ou desistindo; e, se repetir uma ferramenta mutante, duplica o efeito.

NO CASO BALCÃO

`get_data` responde `invalid input`. `consultar_cobrancas` responde: “mes deve ter o formato AAAA-MM, por exemplo '2026-10'. Recebido: 'outubro'. Reenvie com o formato correto.” `registrar_nota` se declara mutante e não idempotente; `emitir_estorno`, destrutiva.

CONFUSÃO FREQUENTE

Tratar o erro como mensagem para humano. Num agente, o primeiro leitor do erro é o modelo.

PARA CONFERIR · RESPOSTA NO FIM

O agente chamou `registrar_nota`, a conexão caiu antes da resposta e ele chamou de novo. O que pode ter acontecido no Balcão?

Conceito 5 de 5

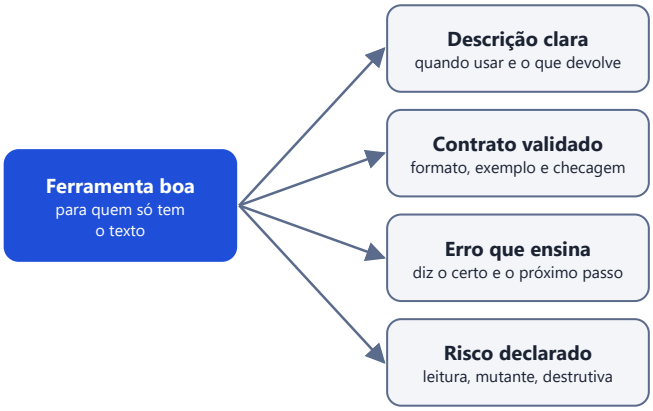
Escreva a ferramenta para um colega que chegou hoje

> **Ferramenta boa é a que um colega novo usaria sem perguntar: descrição clara, contrato validado, erro que ensina e risco declarado.**

A régua da Anthropic é descrever a ferramenta como para uma pessoa que entrou no time hoje: ela não sabe o formato dos ids, a ordem em que as ferramentas se encaixam nem o que é perigoso repetir. O modelo está nessa posição em toda chamada. Até o nome do campo conta: `user_id` diz o que esperar; `user`, não.

As quatro propriedades vêm dos conceitos anteriores: descrição clara (quando usar, o que devolve); contrato validado (formato e exemplo no esquema, conferência no código, saída que encaixa); erro que ensina; risco declarado.

Ferramentas ruins costumam nascer como espelho de uma API interna. A recomendação é partir da tarefa do agente: o exemplo da Anthropic troca `list_users`, `list_events` e `create_event` por uma `schedule_event`.



Síntese em leque: a ferramenta boa, escrita para quem só tem o texto, tem quatro propriedades: descrição clara, que diz quando usar e o que devolve; contrato validado, com formato, exemplo e checagem; erro que ensina, dizendo o certo e o próximo passo; e risco declarado: leitura, mutante ou destrutiva. O teste: um colega que chegou hoje usaria sem perguntar? O modelo também.

NO CASO BALCÃO

Passe o `get_data` pela régua: "Consulta dados." não diz quando usar; `cliente` e `periodo` não dizem formato; `invalid input` não ensina. As ferramentas do contrato forte passam nos quatro critérios.

CONFUSÃO FREQUENTE

Achar que descrição longa é descrição boa. O que conta é ter o que o colega novo precisaria; o resto ocupa contexto em toda chamada.

PARA CONFERIR · RESPOSTA NO FIM

Uma ferramenta `apagar_ticket(ticket_id)` tem a descrição “Apaga o ticket.” e o erro “not found”. O que falta?

Glossário da aula

TERMO	O QUE É
Ferramenta (tool)	função que o modelo pode pedir; ele a conhece pelo nome, pela descrição e pelo esquema de entrada.
JSON Schema	formato que descreve os campos, os tipos e as regras da entrada de uma ferramenta.
Function calling	o ciclo em que o modelo pede uma ferramenta e o programa executa e devolve o resultado; também chamado tool use.
`tool_use`	bloco em que o modelo pede uma ferramenta: `name`, `input` e `id`.
`tool_result`	bloco com o resultado ou o erro, ligado ao pedido por `tool_use_id`.
Contrato	acordo de entrada e saída da ferramenta: esquema, validação no código e formato da resposta.
Validação	conferência, no código, de que os argumentos estão no formato e fazem sentido antes de executar.
Erro que ensina	mensagem com o esperado, o recebido e o que fazer.
Idempotente	operação que, repetida com os mesmos argumentos, tem o mesmo efeito de uma vez só.
Leitura, mutante, destrutiva	classes de risco: só consulta; muda estado; faz algo que não se desfaz.
MCP	protocolo que liga um servidor de ferramentas a um cliente como o Claude Code; tema da aula 3.
`mcp__<servidor>__<ferramenta>`	nome de uma ferramenta MCP dentro do Claude Code, usado em permissões e hooks.

Prática 1 · Rode os dois contratos e ache cada par no registro

45 min

Objetivo. Ver o que cada contrato publica, rodar o mesmo ticket com os dois e casar, no registro, cada `tool_use` com o seu `tool_result`.

1. Veja, sem chamar o modelo, o que o contrato fraco publica: nome, descrição, esquema e anotações. É exatamente o que o modelo recebe. Troque `fraco` por `forte` e rode de novo.

```
python -c "import asyncio, servidor_balcao as s; s.registrar_ferramentas('fraco');
[print(t.model_dump_json(indent=2, exclude_none=True)) for t in
asyncio.run(s.mcp.list_tools())]"
```

2. Rode o agente no TCK-002 com cada contrato. O script grava em `..\estado\config\mcp.json` o comando que liga o servidor ao Claude Code: a tomada que a aula 3 abre.

```
python rodar_agente.py --ticket TCK-002 --modo mcp --contrato fraco --saida fraco.jsonl
python rodar_agente.py --ticket TCK-002 --modo mcp --contrato forte --saida forte.jsonl
```

3. Compare as duas execuções em uma linha cada: voltas, ferramentas chamadas, chamadas com erro e custo.

```
python ler_execucao.py fraco.jsonl --json
python ler_execucao.py forte.jsonl --json
```

4. Abra `forte.jsonl` num editor e busque `tool_use`. Para cada pedido, anote o fim do `id`, o `name` e o `input`; ache o `tool_result` com o mesmo valor em `tool_use_id` e anote `is_error` e o começo do conteúdo. Faça o mesmo em `fraco.jsonl`.

5. Confira do lado do servidor: cada linha é uma chamada que o código executou, com os argumentos que chegaram e o resultado ou o erro.

```
Get-Content ..\estado\chamadas.jsonl
```

O QUE OBSERVAR

- Os nomes chegam como `mcp__balcao__...`: servidor e ferramenta. Todo `tool_use` tem um `tool_result` com o mesmo id.
- No contrato fraco, o que o modelo fez depois de `invalid input`.
- No contrato forte, se o `cliente_id` devolvido por `buscar_cliente` aparece no `input` da chamada seguinte.

ESTÁ PRONTO QUANDO

Você tem, para as duas execuções, a tabela de pares `tool_use` e `tool_result` e uma frase sobre o que mudou com o contrato.

PARA IR ALÉM

Peça ao Claude Code, aberto em `caso-balcao`, a mesma tabela e compare com a sua: “Leia ferramentas/forte.jsonl e monte uma tabela com cada tool_use (id, name, input) e o tool_result correspondente (tool_use_id, is_error, início do conteúdo). Não altere nenhum arquivo.”

Prática 2 · Crie uma ferramenta com contrato e teste antes de ligar

75 min

Objetivo. Escrever uma ferramenta nova no servidor do Balcão, com descrição, validação, erro que ensina e risco declarado, testá-la com pytest sem o modelo e só depois ligá-la ao agente.

1. Na pasta que contém `caso-balcao`, faça uma cópia e trabalhe só nela: o original fica intacto, e o `rodar_agente.py` da cópia já liga o agente à sua versão.

```
Copy-Item -Recurse caso-balcao caso-balcao-p2
cd caso-balcao-p2
```

2. Escreva o contrato antes do código, para `verificar_duplicidade(cliente_id, mes)`: descrição (quando usar, o que devolve), formato e exemplo de cada campo, a mensagem de cada erro e a classe de risco.

3. Abra o Claude Code na cópia (`claude`) e peça a ferramenta e os testes com este texto:

> **TEXTO PARA O CLAUDE CODE**
>
> Em ferramentas/servidor_balcao.py, acrescente ao contrato forte uma ferramenta de leitura chamada verificar_duplicidade(cliente_id, mes). Ela devolve os grupos de lançamentos do cliente no mês com mesma data, tipo e valor, cada grupo com os ids dos lançamentos, e uma lista vazia quando não houver duplicidade. Siga o padrão de consultar_cobrancas: decorador @registrada, parâmetros com Annotated e Field com formato e exemplo, validação com expressão regular e mensagens de erro que digam o esperado, o recebido e o que fazer (por exemplo, chamar buscar_cliente quando vier um nome no lugar do id). Registre a ferramenta em registrar_ferramentas, só no contrato forte, com uma descrição que diga quando usar e o que devolve, e com a anotação LEITURA. Depois crie ferramentas/test_verificar_duplicidade.py, com pytest, cobrindo: CLI-102 em 2026-10 devolve um grupo com COB-5002 e COB-5003; CLI-101 em 2026-10 devolve lista vazia; um nome no lugar do id gera ValueError cuja mensagem cita buscar_cliente; mes 'outubro' gera ValueError cuja mensagem cita o formato AAAA-MM. Chame a função com argumentos nomeados. Não altere as outras ferramentas.

4. Antes de aceitar, compare a mudança com o contrato do passo 2: a descrição diz quando usar? Cada erro tem o esperado, o recebido e o que fazer? A ferramenta entrou só no contrato forte, como leitura?

5. Rode os testes. Eles chamam a função direto, sem modelo: rápidos, gratuitos e sempre com o mesmo resultado. Depois troque, no editor, a mensagem de erro de `cliente_id` por `invalid input` e rode de novo: o teste da mensagem deve falhar. Desfaça a troca.

```
cd ferramentas
python -m pytest test_verificar_duplicidade.py -q
```

6 Ligue a ferramenta ao agente com um pedido livre sobre o Bazar Estrela, que também tem mensalidade duplicada. No resumo, veja qual ferramenta ele escolheu, a sua ou `consultar_cobrancas`, e com que argumentos.

```
python rodar_agente.py --modo mcp --contrato forte --saida p2.jsonl --pedido "Você é o agente de suporte do Balcão. O Bazar Estrela diz que a mensalidade de outubro de 2026 foi cobrada duas vezes. Confira com as ferramentas do Balcão e diga quais lançamentos se repetem. Não emita estorno nem registre nota."
```

O QUE OBSERVAR

- Os testes rodam sem modelo e sem custo; o agente, não. E o teste da mensagem protege o contrato: se alguém deixar o erro opaco, ele falha.
- Os testes também gravam em `estado/chamadas.jsonl` da cópia: quem registra é o código, não o modelo.
- A escolha entre a sua ferramenta e `consultar_cobrancas` depende das descrições das duas.

ESTÁ PRONTO QUANDO

Os testes passam, o teste da mensagem falha quando o erro fica opaco, e o registro do agente mostra a sua ferramenta disponível e, se ele a escolheu, chamada com argumentos válidos.

PARA IR ALÉM

Troque a descrição da sua ferramenta por “Verifica coisas.” e rode o mesmo pedido. O agente ainda a escolhe? Relacione com o conceito 1.

Prática 3 · Piore e melhore uma mensagem de erro 50 min

Objetivo. Medir, com o mesmo pedido, como o agente se recupera de um argumento errado quando o erro ensina, quando é opaco e quando você o melhora.

1. Na pasta que contém `caso-balcao`, faça outra cópia, entre em `ferramentas` e limpe o registro do servidor.

```
Copy-Item -Recurse caso-balcao caso-balcao-p3
cd caso-balcao-p3\ferramentas
Remove-Item ..\estado\*.jsonl -ErrorAction SilentlyContinue
```

2. Guarde numa variável, no mesmo terminal, um pedido que força a primeira chamada a sair errada, e rode com as mensagens originais.

```
$pedido = "Você é o agente de suporte do Balcão. Ticket TCK-002, Estúdio Pilates Vida:
a mensalidade de outubro foi cobrada duas vezes. Comece chamando consultar_cobrancas
com cliente_id 'Estúdio Pilates Vida' e mes 'outubro', exatamente assim. Depois
continue com as ferramentas do Balcão e diga se houve duplicidade. Não emita estorno
nem registre nota."
python rodar_agente.py --modo mcp --contrato forte --saida ensina.jsonl --pedido
$pedido
```

3. No resumo, ache a linha `ERRO` e a chamada seguinte: o agente seguiu a mensagem? Quantas chamadas até `consultar_cobrancas` dar certo? Se ele foi direto a `buscar_cliente`, anote: a descrição do campo pesou mais que o pedido.

4. Piore as mensagens. Abra o Claude Code em `caso-balcao-p3` e peça:

> TEXTO PARA O CLAUDE CODE
> Em ferramentas/servidor_balcao.py, na função consultar_cobrancas, troque o texto das duas mensagens de erro de formato (a de cliente_id e a de mes) por exatamente 'invalid input'. Não mude mais nada.

5. Rode o mesmo pedido. Depois melhore: escreva no editor uma mensagem para o erro de `cliente_id` melhor que a original (por exemplo, com um exemplo de chamada a `buscar_cliente`), devolva a de `mes` ao texto original e rode de novo.

```
python rodar_agente.py --modo mcp --contrato forte --saida opaco.jsonl --pedido $pedido
python rodar_agente.py --modo mcp --contrato forte --saida melhor.jsonl --pedido
$pedido
```

6. Compare as três execuções em uma linha cada.

```
python ler_execucao.py ensina.jsonl --json
python ler_execucao.py opaco.jsonl --json
python ler_execucao.py melhor.jsonl --json
```

O QUE OBSERVAR

- Com `invalid input`, o agente pode tentar outros valores, desistir ou se recuperar mesmo assim. Se ele se recuperou, a hipótese mais provável é que a descrição do campo que falhou (`cliente_id` ou `mes`) ensinou o formato: conceito 1. Nas execuções do guia, foi o que aconteceu com o `mes`.
- Cada tentativa extra aparece como volta e como custo; em `..\estado\chamadas.jsonl`, a sequência de argumentos das três execuções.

ESTÁ PRONTO QUANDO

Você tem uma tabela com as três execuções (chamadas com erro, voltas, se o agente se recuperou, custo) e uma frase sobre o que uma mensagem de erro precisa conter.

PARA IR ALÉM

Tire também a dica da descrição do campo `cliente_id` ("Se você só tem o nome, chame buscar_cliente antes.") e rode a versão opaca de novo. Agora só o erro pode ensinar.

Autoavaliação

Marque o que você já consegue fazer sem consultar o material. O que ficar em branco indica o conceito para reler.

- [ ] Consigo dizer as três partes que o modelo recebe de uma ferramenta e por que ele nunca vê o código.
- [ ] Consigo descrever o ciclo `tool_use`, execução e `tool_result` e o papel do id.
- [ ] Consigo explicar por que validar no código mesmo com esquema e o que é uma saída que encaixa no próximo passo.
- [ ] Consigo reescrever um erro opaco como erro que ensina.
- [ ] Consigo classificar uma ferramenta como leitura, mutante ou destrutiva e dizer o que acontece se ela for repetida.
- [ ] Consigo avaliar uma ferramenta pelas quatro propriedades da síntese.

Respostas das perguntas

1. O TCK-002 traz o nome do comércio, não o id. Com o contrato fraco, de onde o modelo tiraria o formato do campo `cliente`?
De lugar nenhum: nem a descrição nem o esquema dizem o formato. Ele só pode adivinhar.

2. Se o modelo pedir duas ferramentas na mesma volta, como o seu código diz qual resultado é de qual pedido?
Pelo id: cada `tool_result` leva o `tool_use_id` do pedido a que responde.

3. Por que `buscar_cliente` devolve só id, nome, plano e mensalidade, e não o cadastro inteiro?
Porque o próximo passo precisa do id, e o resto do que ela devolve ajuda a conferir o caso. Mais campos ocupariam a janela em todas as voltas seguintes.

4. O agente chamou `registrar_nota`, a conexão caiu antes da resposta e ele chamou de novo. O que pode ter acontecido no Balcão?
Duas notas iguais no ticket: a ferramenta é mutante e não idempotente, e a primeira chamada pode ter sido gravada antes da falha.

5. Uma ferramenta `apagar_ticket(ticket_id)` tem a descrição "Apaga o ticket." e o erro "not found". O que falta?
A descrição não diz quando usar nem o formato de `ticket_id`; o erro não ensina o que fazer; e o risco não está declarado, embora ela seja destrutiva.

Para ler depois

- Anthropic, "Writing effective tools for AI agents" (11/09/2025). https://www.anthropic.com/engineering/writing-tools-for-agents — princípios para escrever ferramentas: descrição, consolidação, saída com significado, tamanho e erros.
- Anthropic, tool use na API ("Tool use with Claude"). https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview — o ciclo `tool_use`

e `tool_result`, `tool_choice` e modo estrito.

- Claude Code, MCP. https://code.claude.com/docs/en/mcp — `claude mcp add`, `/mcp` e o limite de saída das ferramentas MCP.
- Claude Code, referência da CLI. https://code.claude.com/docs/en/cli-reference — opções como --mcp-config, --strict-mcp-config, --allowedTools e --disallowedTools.